

Take-home: Harness

ROLE	Agentic AI Engineer
WINDOW	3 days from receipt

The brief in one sentence

Take a capable agent and make it drivable, through a browser, by an operator who will never open a terminal.

Why this assignment

You already proved you can build a deep autonomous agent. We read your repo, counted your tools, verified your subagent isolation and graded your prompts. That box is ticked and we are not asking you to tick it again.

What we have not seen from you is the other half of the job, which is most of our job: putting a serious agent behind an interface that a non-engineer can actually drive, trust and stay in control of.

Think Claude.ai, Claude Code, Codex, opencode. Now think about what those would need to become for someone who does not know what a tool call is, has never seen a terminal, and is going to use it every day to do real work. That is the product.

What you are building

A web app where a human operates an agent that does long, deep, multi-step work.

Build a new agent for this one, in a domain you have not built for us before. Keep the engine lean, because it is not what we are measuring here and it only has to work hard enough to give the harness something real to render.

There is no tool cap and no seed data. The opposite: the harder your agent works, the more interesting this assignment gets. We need a run with real depth to judge how you render it. As a floor, the runs you show us should exceed 20 tool calls, cross several namespaces, and spawn at least one subagent.

The engine is the smaller half of this. The harness is the assignment. Nothing needs to be hosted anywhere, and we will watch it run in your video.

What the harness has to do

You will not build all of this. Pick the ones that matter most and go deep. Say what you cut and why.

1 Make a deep run legible

This is the centre of the assignment and it carries the most weight.

An agent run fires sixty tool calls, spawns subagents that fire twenty more each, and takes eight minutes. A person is watching. At any moment they should be able to answer, at a glance, what is happening right now, what has been done so far, and roughly how far through we are.

Raw tool logs are not the answer. Neither is hiding everything behind a spinner. The interesting design work lives between those two failures: grouping, nesting, summarising, collapsing, deciding what is worth surfacing and what is noise, and letting someone drill in when they want detail.

Subagents are the hard case. When a subagent is running twenty calls of its own inside a parent run, that has to read as a coherent nested thing rather than an interleaved mess.

2 Let the operator stay in control

- Interrupt a run mid-flight.
- Redirect it without throwing away what it has already done.
- Queue a message while it is still working.
- Set how much autonomy it has, in language a non-engineer understands. Claude Code's permission modes are the right idea and the wrong vocabulary for this user.

3 Earn trust before it does something consequential

- The agent proposes an action that writes, spends, sends or deletes. A human approves it inline, without losing their place in the run.
- The approver can edit the proposal before approving, not just accept or reject.
- Show what is about to change before it changes. A preview or a diff, not a promise.
- After the fact, the operator can see what the agent read and what it changed.

Design what happens to the run, and to the person watching it, while an approval is pending.

4 Survive real life

- The operator closes the tab at minute three and comes back at minute nine. The run is still there.
- A tool fails. The model errors. The run dies halfway. Each of those is a designed state, not a stack trace.
- Resume or retry without starting from zero.
- Past sessions are findable and readable later.

5 Deliver the work

Runs produce things: files, reports, diffs, structured results. Getting those to the operator in a usable form is part of the product. A code block is rarely the right answer.

Deliverables

1. **A public GitHub repo** with real commit history.

2. **Your session traces, committed into the repo, in raw form**, under `traces/`. Whatever you coded with (Claude Code, Codex, Cursor, opencode, anything) exports a session log. Commit the actual files. Do not link a Drive folder, do not paste screenshots, do not summarise. We grade how you drive a coding agent, and last round we could not grade a third of the field because their traces were not readable. If we cannot open it, it did not happen. We also check traces for tampering, and last round five candidates were removed for integrity failures, four of them for exactly that, including the highest-scoring repo in the pool. A MEMO that admits something is stubbed always reads better than a polished claim we can disprove.
3. **MEMO.md**, maximum two pages:
 - who you decided the operator is, and what they know and do not know
 - what you cut, and why
 - the hardest rendering decision you made, and what you rejected
 - every failure state, and what the operator sees in it
 - the one metric you would watch in production
 - anything stubbed, faked or hardcoded, disclosed up front
4. **A demo video**: drive a hard run, start to finish, as the operator. Interrupt it. Approve something. Break it on purpose and show what the operator sees. A code walkthrough scores zero. We can read the code. We are watching how you think about the person driving.

A closing note on taste

There is a default look that AI products fall into: dark background, purple gradient, glowing orb, chat bubbles, sparkle icon. If you ship that, we will assume the model chose it and you did not.

Make a decision about how this should look and feel, and defend it in one paragraph in the MEMO. It does not need to be beautiful. It needs to be chosen.

Submit within 3 days of receipt, by reply on the email thread. Questions are welcome.